

CMPSC 473 Project 1 Report

Siddhardh Tekumalla

Part 1: C Pointers and Memory Refresher

1. `p` is type `int *`. It refers to the address of `a[0]`. `pp` is type `int **`. It refers to `p`. `*pp` is type `int *`. It refers to the value at address `p` - aka the address of `a[0]`. `**pp` is type `int`. It refers to the value of `a[0] = 10`.
2. `p` refers to the address of `a[0]`. `&p` is the address of `p`, aka the address of the address of `a[0]`. `*p` is the value stored at the address `p` holds, which is 10. `a` is an integer array of size four, but holds the address of `a[0]`. `&a` refers to the address of the entire array.
3. `p+1` means we move its address by 1 unit forward, meaning it moves from the address of `a[0]` to that of `a[1]`. `p+1` is not always `A+1`, because numeric addresses move in increments of (usually) four.
4. 10, 30, 30
5. `a`: stack. `p`: stack. `pp`: stack. `local`: stack. `heap`: stack. struct pair returned by `malloc`: heap (`malloc` gives it away). Pointer variable `heap` is in the stack, but the struct pair it points to is in the heap.
6. `malloc(sizeof *heap)` is preferable because a manual, numerical allocation is much more error prone. Automatic adjustment is preferable. `sizeof(heap)` measures the size of the address of the struct pair named `heap`. `sizeof(*heap)` measures the size of the struct pair itself.

Part 2: Make and Separate Compilation

1. Running `make` a second time, without modification, leads to nothing being recompiled. This is because everything is up to date, with neither object file older than the `.c` files they're compiled from.
2. After modifying `stats` and running `make`, `stats.o` is rebuilt. This is since now, `stats.o` is older than the `stats.c` file it's compiled from. But with `stats.o` rebuilt, it's now newer than `stats.c` as it should.
3. Modifying `stats.h`, `stats.o` was again rebuilt. This is since `stats.h` can't be newer than `stats.o`, so the latter is rebuilt. The logic is similar to the answer above.

Part 3: Debugging With GDB

Running normally I observed no failure. Code returned total=100. This, however, might be hiding a deeper error. Starting GDB I put a breakpoint in line 13. I used 'continue' to move through iterations. Each time I used commands `print(i)` and `print(records[i])`. I used `x` to examine the memory of the records pointer: `x/4iw records`. As said previously, no overt failure was observed. Regardless, I still used `backtrace` to confirm addresses. The bug is that the iterator `i` goes beyond the array index. Use of `<=` instead of `<` means `i` goes to 4, corresponding to the fifth element of array `records` - which doesn't exist. So the value for that loops around to zero, giving a "correct" answer despite the error. To prevent the possibility of a crash, I changed line 12 to: `for (int i = 0; i < n; i++)`

Evidence from GDB:

Breakpoint 1, total_scores (records=0x5555555592a0, n=4) at debug.c:13

```
13      total += records[i].score;
```

```
(gdb) print(i)
```

```
$5 = 4
```

```
(gdb) print(records[i])
```

```
$6 = {id = 0, score = 0}
```

Memory Diagram:



The Records object is on heap. Other variables are in the stack.

Part 4: Virtual Address Spaces and /proc

1. The addresses in the 0x7ffe range correspond to stack objects, according to smap's results:

7ffe2e1f1000-7ffe2e212000 rw-p 00000000 00:00 0 [stack]

The addresses in the 0x588e range correspond to heap objects, according to smap's results:

588eea2f1000-588eea312000 rw-p 00000000 00:00 0 [heap]

2. Stack addresses decrease as recursion becomes deeper, while heap allocations increase. And yes, I acknowledge this is not a guarantee across the C language.
3. 588eea2f1000-588eea312000 rw-p 00000000 00:00 0 [heap]
7ffe2e1f1000-7ffe2e212000 rw-p 00000000 00:00 0 [stack]
These addresses lie in the expected regions!

4. Heap has virtual mapping size 132kB, Rss 4kB, and Private_Dirty 4kB. Stack has virtual mapping size 132kB, Rss 16kB, and Private_Dirty 16kB:

00007ffe2e1f1000 132 16 16 rw--- [stack]

Reserving a virtual address range doesn't allocate actual physical RAM. Instead, the process gets addresses it's assured that it can use. *Can*, not will. This means a large virtual address range doesn't always correspond to a large amount of physical memory being occupied.

5. brk - adjusts the program break so that the heap can grow.

read - reads from file descriptors.

write - writes the printed addresses.

MAP_PRIVATE - sets the mapping to be 'copy-on-write'.

mmap - maps a file or an anonymous memory into the virtual address space.

Part 5: Memory Errors

The corrected source is in the code. The table and terminal tests are as shown:

Test	Error Category	Offending Statement	Explanation (one sentence)	Fix
Test1	Out-of-bounds access	For (size_t i = 0; i<=n; ++i)	Loop writes a[n], which is one element past the allocation, causing a buffer overflow.	Change offending statement to <i>for (size_t i=0; i<n; ++i)</i>
Test2	Use of uninitialized data	initialize_prefix(a, n - 1); printf("test2: %d\n", a[0] + a[1] + a[n - 1]);	Only the first <i>n-1</i> elements are initialized, yet a[n-1] is read which causes undefined behavior.	<i>Initialize_prefix(a,n)</i> . Initializing all the elements solves the issue.
Test3	Use after <i>free</i>	release_int(p); printf("test3: %d\n", *p);	P is freed but then dereferences, creating a chance of a crash.	Use the p value before freeing it: <i>printf("test3: %d\n", *p); release_int(p);</i>
Test4	Memory leak	char *message = make_message(); printf("test4: %s\n", message);	Make_message uses heap memory that's never freed, causing a memory leak.	Free <i>message</i> after using it: <i>free(message)</i>

Used the following terminal command:

```
valgrind --leak-check=full --show-leak-kinds=all --track-origins=yes ./memory-bugs  
testnum
```

Test 1:

```
==4769== Invalid write of size 4
```

```
==4769==   at 0x109345: test1 (memory-bugs.c:46)
```

```
==4769==   by 0x10953E: main (memory-bugs.c:92)
```

```
==4769== Address 0x4a8c050 is 0 bytes after a block of size 16 alloc'd
```

Test 2:

==35700== Conditional jump or move depends on uninitialised value(s)

==35700== at 0x48D6A96: __vfprintf_internal (vfprintf-internal.c:1516)

==35700== by 0x48C079E: printf (printf.c:33)

==35700== by 0x109419: test2 (memory-bugs.c:60)

==35700== by 0x109545: main (memory-bugs.c:93)

==35700== Uninitialised value was created by a heap allocation

Test 3:

==35856== Invalid read of size 4

==35856== at 0x10946E: test3 (memory-bugs.c:72)

==35856== by 0x10954C: main (memory-bugs.c:94)

==35856== Address 0x4a8c040 is 0 bytes inside a block of size 4 free'd

Test 4:

==35944== LEAK SUMMARY:

==35944== definitely lost: 64 bytes in 1 blocks

Part 6: Processes, fork, and exec

1. PARENT_AFTER_FORK pid=53678 child=53679 global=0x5c19c4742010
stack=0x7ffebddb75a4 heap=0x5c19f39b42a0 values=100,200,300
CHILD_AFTER_FORK pid=53679 ppid=53678 global=0x5c19c4742010
stack=0x7ffebddb75a4 heap=0x5c19f39b42a0 values=100,200,300
As we can see, the child PID is the parent PID + 1 (the actual numbers themselves vary by execution). Both have the same global address, heap, and stack location. This does not, however, imply both are using the same physical memory location. The reason is because after fork(), the child gets a copy-on-write duplicate of the parent's virtual address space, not the same physical pages. So as soon as either the parent or child writes, the memory locations will diverge.
2. As said above, while the parent and child have identical virtual addresses they have different physical memory once either process writes. This is the basis of copy-on-write. As such, a variable in the child does not affect the parent. Thus, the corresponding value in the parent does not change,
3. The first change is that the program location changed from /processes/parent to /processes/child, telling us that exec was indeed successful. We also saw a change in the heap and stack memory addresses. In total, the old program's code, data, heap, and stack are all gone. The new program of the child now has its own mappings unique from the parent process.
4. I used strace -f -o trace.txt ./parent. The output file should be in the same folder in the repo but I'll continue my answer nonetheless.
The fork/clone-related call is: 61510 clone(child_stack=NULL, flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD, child_tidptr=0x716de1e0fa10) = 61511
The exec call is: 61511 execve("./child", ["child"], 0x7ffdd9cdf258 /* 26 vars */) = 0.
The parent's wait-related call is: 61510 wait4(61511, <unfinished ...>
and is ended at: 61510 <... wait4 resumed>[{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0, NULL) = 61511